فصل ۶: مدل‌ها با معیار فاصله

مقدمه

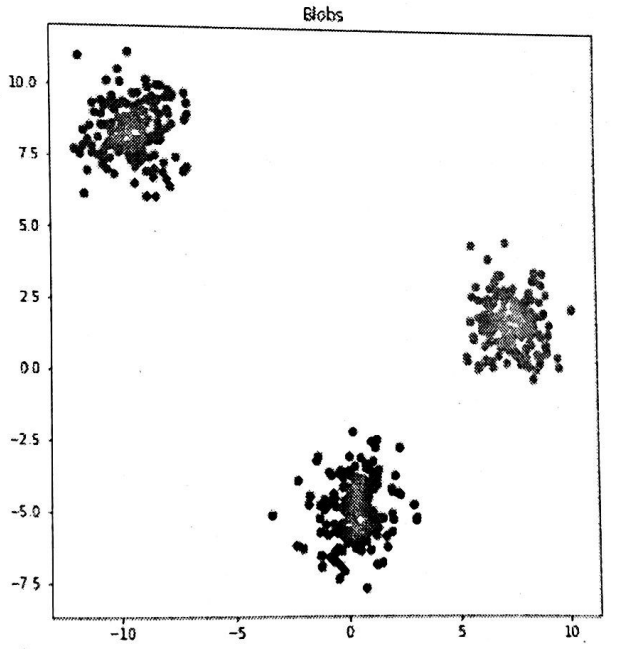
در این فصل، خوشه‌بندی را توضیح خواهیم داد. خوشه‌بندی در روش‌های بدون نظارت قرار داده می‌شود. این روش‌ها فرض می‌کنند که ما از متغیر نتیجه آگاهی (و برچسب داده وجود ندارد) نداریم. این امر در عمل به ابهام در نتایج و اهداف منجر می‌شود، با این وجود خوشه‌بندی دارای مزیت‌هایی می‌باشد. همانطور که خواهیم دید، می‌توانیم از خوشه‌بندی برای محلی‌سازی تخمین خود در یک محیط نظارت شده استفاده کنیم. شاید به همین دلیل است که خوشه‌بندی بسیار موثر است. این روش می‌تواند طیف وسیعی از موقعیت‌ها را کنترل کند. ما در این فصل از طیف گسترده‌ای از برنامه‌ها، از پردازش تصویر گرفته تا رگرسیون و تشخیص دورتر، را مرور خواهیم کرد. خوشه‌بندی به طبقه‌بندی دسته‌ها مربوط می‌شود. شما یک مجموعه محدود از دسته‌ها دارید، و برخلاف کلاسه‌بندی، شما برچسب کلاس‌ها را نمی‌دانید.

استفاده از k-means برای خوشه‌بندی داده‌ها

در یک مجموعه داده، ما مجموعه‌ای از نقاط را در کنار هم قرارمی دهیم. با k-means، همه داده‌ها را به خوشه‌ها دسته‌بندی می‌کنیم.

در اینجا یک مثال ساده که مجموعه‌ای از داده‌ها را خوشه‌بندی می‌کند، را به شما نشان خواهیم داد. سپس کمی در مورد نحوه کار k-means برای یافتن تعداد مطلوب خوشه‌ها صحبت خواهیم کرد.

```
import numpy as np
import pandas as pd
from sklearn.datasets import make_blobs
blobs, classes = make_blobs(500, centers=3)
import matplotlib.pyplot as plt
%matplotlib inline #Within an ipython notebook
f, ax = plt.subplots(figsize=(7.5, 7.5))
rgb = np.array(['r', 'g', 'b'])
ax.scatter(blobs[:, 0], blobs[:, 1], color=rgb[classes])
ax.set_title("Blobs")
```

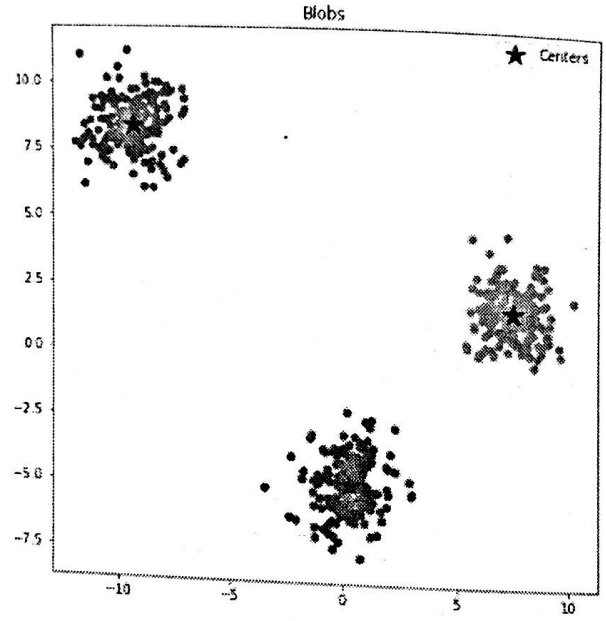


شکل (۶-۱): مجموعه داده برای خوشه‌بندی

اکنون می‌توانیم از k-means برای یافتن مراکز این خوشه‌ها استفاده کنیم. در مثال اول، وانمود خواهیم کرد که می‌دانیم سه مرکز وجود دارد:

```
from sklearn.cluster import KMeans
kmean = KMeans(n_clusters=3)
kmean.fit(blobs)
KMeans(algorithm='auto', copy_x=True, init='k-means++',
max_iter=300,
```

```
n_clusters=3, n_init=10, n_jobs=1, precompute_distances='auto',
random_state=None, tol=0.0001, verbose=0)
kmean.cluster_centers_
output: array([[-3.96755821, -7.06110388],
   [ 2.73308694, -7.37088756],
   [-0.77243196, -5.67214339]])
f, ax = plt.subplots(figsize=(7.5, 7.5))
ax.scatter(blobs[:, 0], blobs[:, 1], color=rgb[classes])
ax.scatter(kmean.cluster_centers_[:, 0],kmean.cluster_centers_[:,
1], marker='*', s=250,color='black', label='Centers')
ax.set_title("Blobs")
ax.legend(loc='best')
```



شکل (۶-۲): نمایش مرکز خوشه‌ها

ویژگی‌های دیگر نیز می‌توانند مفید باشند. به عنوان مثال، ویژگی labels برچسب مورد انتظار برای هر نقطه را تولید می‌کند

```
kmean.labels_[:5]
output: array([0, 0, 0, 0, 1])
```

ما می‌توانیم بررسی کنیم که kmean.labels_ همان کلاس‌ها است یا خیر، اما از آنجا که k-mean هیچ اطلاعی از کلاس‌ها ندارد، نمی‌تواند مقادیر شاخص نمونه را به هر دو کلاس اختصاص دهد:

```
classes[:5]
output: array([1, 1, 1, 1, 2])
```

تابع تبدیل فاصله بین هر نقطه و مرکز را تولید می‌کنیم

```
kmean.transform(blobs)[:5]
output: array([[2.05991436, 7.14091203, 4.84879319],
  [1.02618435, 6.66929343, 3.94840116],
  [2.20424291, 8.06945351, 5.46893779],
  [0.3754817 , 6.81035514, 3.42676317],
  [6.20193385, 0.88654676, 3.7684229 ]])
```

k-mean در واقع یک الگوریتم بسیار ساده است که برای به حداقل رساندن مجموع خوشه‌های مربعات فاصله از میانگین کار می‌کند. ما دوباره جمع مربعات را به حداقل می‌رسانیم. روند کار به صورت زیر است:

- تعیین تعداد خوشه‌ها
- اختصاص دادن هر داده به نزدیک‌ترین خوشه
- به‌روزرسانی مرکز هر خوشه با محاسبه میانگین مشاهدات اختصاص داده شده به این خوشه

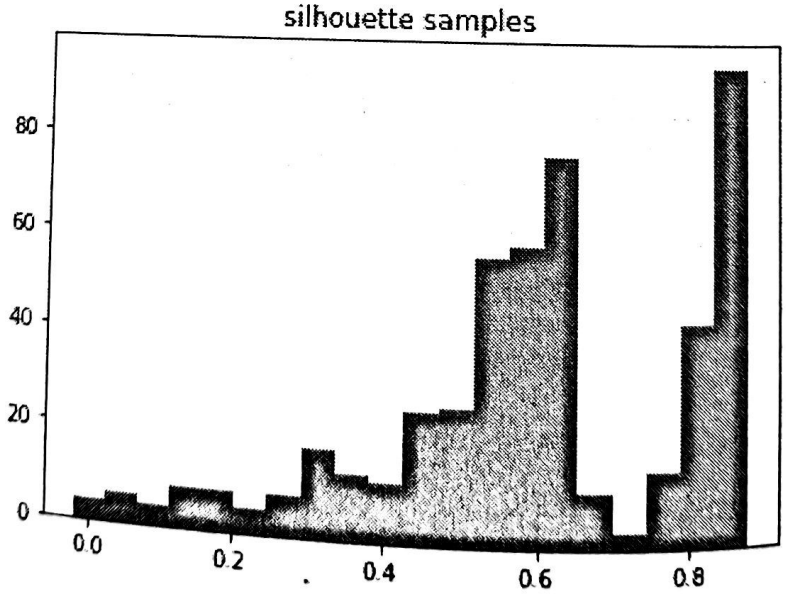
این روند تکرار می‌شود تا زمانی که برخی از معیارهای مشخص شده برآورده شود. تفسیر مرکز خوشه دشوار است و همچنین تعیین اینکه آیا تعداد صحیح مرکز خوشه داریم یا نه نیز بسیار دشوار است. مهم است که درک کنید آیا داده‌های شما بدون برچسب هستند یا خیر، زیرا این امر مستقیماً بر اقدامات ارزیابی که می‌توانید استفاده کنید تأثیر می‌گذارد.

بهینه‌سازی تعداد مرکز خوشه‌ها

هنگام انجام خوشه‌بندی k-means، ما تعداد دقیق خوشه‌ها را از قبل نمی‌دانیم، بنابراین کشف این یک مرحله مهم است. هنگامی که از تعداد مرکز خوشه‌ها مطلع شدیم (یا تخمین زدیم)، این بیشتر شبیه یک کلاسه‌بندی خواهد بود زیرا دانش ما از آن بسیار افزایش خواهد یافت. ارزیابی عملکرد مدل برای تکنیک‌های بدون نظارت یک چالش است. در نتیجه، sklearn روش‌های مختلفی برای ارزیابی خوشه‌بندی دارد. ما با یک مدل خوشه‌ای واحد شروع می‌کنیم و شباهت آن را ارزیابی می‌کنیم. برای این کار: چندین نقطه ایجاد خواهیم کرد که می‌تواند برای شبیه‌سازی خوشه داده‌ها مورد استفاده قرار گیرد.

ابتدا، ما به فاصله silhouette نگاه خواهیم کرد. فاصله silhouette، اختلاف بین عدم تشابه درون خوشه و نزدیک‌ترین عدم تشابه خارج خوشه را اندازه می‌گیرد و سپس حداکثر این دو مقدار را به عنوان جواب انتخاب می‌کند. می‌توان آن را به عنوان معیاری برای جدا بودن خوشه‌ها در نظر گرفت. بیایید توزیع فواصل از نقاط تا مراکز خوشه را بررسی کنیم.

```
from sklearn.datasets import make_blobs
import numpy as np
blobs, classes = make_blobs(500, centers=3)
from sklearn.cluster import KMeans
kmean = KMeans(n_clusters=3)
kmean.fit(blobs)
KMeans(algorithm='auto', copy_x=True, init='k-means++',
max_iter=300,
n_clusters=3, n_init=10, n_jobs=1,
precompute_distances='auto',
random_state=None, tol=0.0001, verbose=0)
from sklearn import metrics
silhouette_samples = metrics.silhouette_samples(blobs, kmean.labels_)
np.column_stack((classes[:5], silhouette_samples[:5]))
output: array([[1.   , 0.25621935],
   [1.   , 0.2639256 ],
   [0.   , 0.82849555],
   [2.   , 0.18937469],
   [1.   , 0.44786238]])
plt.hist(silhouette_samples , bins=20)
plt.title("silhouette samples")
plt.show()
```

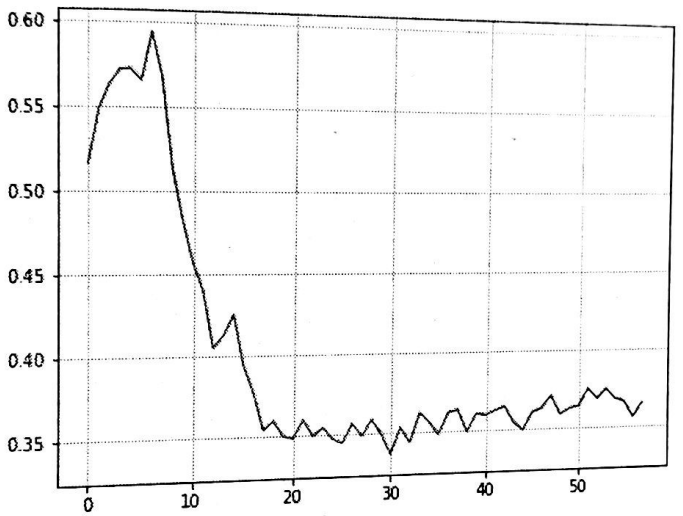


شکل (۶-۳): نمودار ستونی silhouette

توجه داشته باشید که به طور کلی، هرچه تعداد ضرایب نزدیک به ۱ افزایش یابد (که خوب است)، بهتر است.

از میانگین ضرایب silhouette اغلب برای توصیف تناسب کل مدل استفاده می‌شود.

```
silhouette_samples.mean()
output: 0.5990064749152082
blobs, classes = make_blobs(500, centers=10)
silhouette_avgs = []
for k in range(2, 60):
 kmean = KMeans(n_clusters=k).fit(blobs)
 silhouette_avgs.append(metrics.silhouette_score(blobs, kmean.labels_))
f, ax = plt.subplots(figsize=(7, 5))
ax.plot(silhouette_avgs)
```



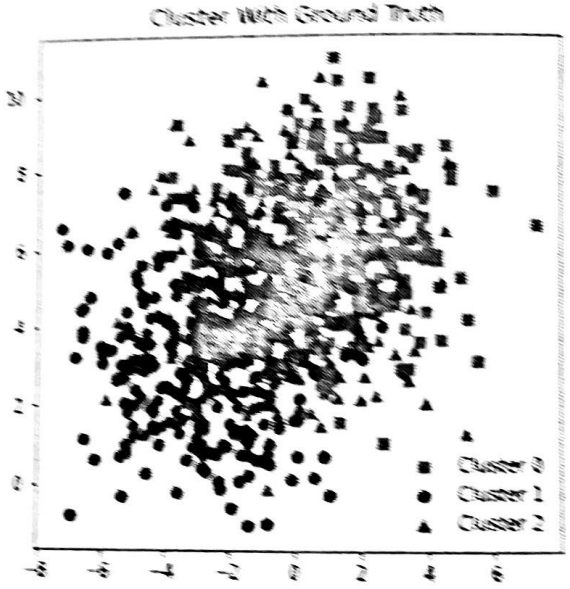
شکل (۶-۴): نمودار میانگین sillouette

این نمودار نشان می‌دهد که با افزایش تعداد مراکز، silhouette به طور متوسط شکل می‌گیرد. می‌توانیم ببینیم که با توجه به فرآیند تولید داده، عدد بهینه ۳ است. اما در اینجا به نظر می‌رسد که حدود ۷ یا ۸ است. این واقعیت خوشه‌بندی است. اغلب اوقات، تعداد درستی از خوشه‌ها بدست نخواهیم آورد: ما واقعاً می‌توانیم امیدوار باشیم که تعداد خوشه‌ها را تا حدودی تخمین بزنیم.

ارزیابی صحت خوشه

ما کمی در مورد ارزیابی خوشه‌ها صحبت کردیم زمانی که تعداد خوشه‌ها مشخص نیست. با این حال، ما هنوز در مورد ارزیابی k-means هنگامی که خوشه شناخته شده است صحبت نکرده‌ایم. ما یک مجموعه داده ساده ایجاد خواهیم کرد، خوشه‌بندی می‌کنیم و سپس در مورد صحت آن بحث خواهیم کرد

```
from sklearn import datasets
from sklearn import cluster
blobs, ground_truth = datasets.make_blobs(1000, centers=3,cluster_std=1.75)
%matplotlib inline
import matplotlib.pyplot as plt
f, ax = plt.subplots(figsize=(7, 5))
colors = ['r', 'g', 'b']
for i in range(3):
  p = blobs[ground_truth == i]
  ax.scatter(p[:,0], p[:,1], c=colors[i],
  label="Cluster {}".format(i))
ax.set_title("Cluster With Ground Truth")
ax.legend()
```



شکل (۶-۲): نمونه داده‌ها برای خوشه‌بندی

```
kmeans = cluster.KMeans(n_clusters=3)
kmeans.fit(blobs)
KMeans(algorithm='auto', copy_x=True, init='k-means++',
max_iter=300,
n_clusters=3, n_init=10, n_jobs=1,
precompute_distances='auto',
random_state=None, tol=0.0001, verbose=0)
kmeans.cluster_centers_
output: array([[ 0.54260598, 6.90156826],
   [ 3.48520748, -6.62479104],
   [-2.75675024, 5.83011789]])
f, ax = plt.subplots(figsize=(7, 5))
```

```
colors = ['r', 'g', 'b']
for i in range(3):
 p = blobs[ground_truth == i]
 ax.scatter(p[:,0], p[:,1], c=colors[i], label="Cluster {}".format(i))
from sklearn import metrics
metrics.normalized_mutual_info_score(ground_truth, kmeans.labels_)
output: 0.6655375933920158
```

اینرسی

یکی از معیارهای خوشه‌ای که ما هنوز در مورد آن صحبت نکرده‌ایم و متکی به حقیقت نیست، اینرسی است. در حال حاضر به عنوان متریک خیلی خوب مستند نیست. با این حال معیاری است که k-mean به حداقل می‌رساند. اینرسی مجموع اختلاف مربع بین هر نقطه و خوشه اختصاص داده شده آن است.

```
kmeans.inertia_
output: 4817.0869756691845
```

استفاده از Mini Batch k-means برای مدیریت داده‌های بیشتر

K-mean یک روش خوب برای خوشه‌بندی است. با این حال، برای بسیاری از داده‌ها ایده آل نیست. این به دلیل پیچیدگی k-means است. با این اوصاف، ما می‌توانیم با استفاده از MiniBatch k-means راه حل‌های تقریبی با پیچیدگی الگوریتمی بسیار بهتری بدست آوریم. MiniBatch k-mean اجرای سریع‌تر k-means است. K-mean از نظر محاسباتی بسیار گران است. از جمله مسائل NP سخت است. با این حال، با استفاده از MiniBatch k-means، می‌توانیم سرعت k-means را به ترتیب بزرگنمایی افزایش دهیم. این هدف با در نظر گرفتن بسیاری از نمونه‌های کوچک که MiniBatches نامیده می‌شوند، حاصل می‌شود.

ابتدا، ما اختلاف کلی را بررسی خواهیم کرد و سپس خطاهای تخمین‌ها را بررسی خواهیم کرد. این معیارها برای افشای مسئله است. بنابراین، برای اطمینان از بالاترین دقت آزمون کارایی (benchmark) دقت زیادی صورت می‌گیرد. اطلاعات زیادی در مورد این موضوع در دسترس است. اگر واقعاً می‌خواهید بدانید که چرا MiniBatch k-means در مقیاس‌بندی بهتر است، بهتر است موارد موجود را مرور کنید.

```
import numpy as np
from sklearn.datasets import make_blobs
blobs, labels = make_blobs(int(1e6), 3)
```

```
from sklearn.cluster import KMeans, MiniBatchKMeans
kmeans = KMeans(n_clusters=3)
minibatch = MiniBatchKMeans(n_clusters=3)
```

ما می‌توانیم اختلاف زمان را اندازه‌گیری کنیم

```
%time kmeans.fit(blobs) #IPython Magic

KMeans(algorithm='auto', copy_x=True, init='k-means++',
max_iter=300,
n_clusters=3, n_init=10, n_jobs=1, precompute_distances='auto',
random_state=None, tol=0.0001, verbose=0)
%time minibatch.fit(blobs)

MiniBatchKMeans(batch_size=100, compute_labels=True, init='kmeans++',
init_size=None, max_iter=100,
max_no_improvement=10,n_clusters=3,n_init=3, random_state=None,
reassignment_ratio=0.01,tol=0.0,verbose=0)
output: Wall time: 6.19 s
Wall time: 3.07 s
```

نکته:

تفاوت زیادی در زمان پردازنده وجود دارد.

حال به مراکز انتخاب شده توجه کنیم

```
kmeans.cluster_centers_
output:array([[-4.33932654, -6.31282509, 4.25621902],
   [-4.43339849, 5.75853411, -5.08316411],
   [ 3.14086974, 3.20399398, 5.93645384]])
minibatch.cluster_centers_
output:array([[-4.33563062, -6.29733747, 4.27481043],
   [-4.39235785, 5.74296539, -5.09934358],
   [ 3.16546018, 3.226453 , 5.96154074]])
```

به فاصله بین اولین مراکز خوشه نگاه کنید (بین kmeans معمولی و minibatch) به نظر می‌رسد که آن دو تقریبا نزدیک به هم انتخاب شده‌اند. قطر شامل اختلافات مرکز خوشه خواهد بود:

```
from sklearn.metrics import pairwise
pairwise.pairwise_distances(kmeans.cluster_centers_[0].reshape(1,
-1), minibatch.cluster_centers_[0].reshape(1, -1))
Output: array([[0.02447788]])
np.diag(pairwise.pairwise_distances(kmeans.cluster_centers_,minibatch.cluster
_centers_))
output: array([0.02447788, 0.04678135, 0.04169472])
```

در اینجا دسته‌ها نقش کلیدی دارند. تکرار دسته‌ها برای یافتن میانگین آن‌ها انجام می‌شود. برای تکرار بعدی، میانگین دسته قبلی در رابطه با تکرار فعلی به‌روز می‌شود. گزینه‌های مختلفی برای رفتار کلی k-means و پارامترها وجود دارد که تعیین می‌کند MiniBatch - چگونه به‌روز می‌شود. پارامتر batch_size تعیین می‌کند که دسته‌ها باید چه اندازه‌ای داشته باشند.

```
minibatch = MiniBatchKMeans(batch_size=len(blobs))
%time minibatch.fit(blobs)
MiniBatchKMeans(batch_size=1000000, compute_labels=True, init='k-
means++',
init_size=None, max_iter=100, max_no_improvement=10, n_clusters=8,
n_init=3, random_state=None, reassignment_ratio=0.01, tol=0.0,
verbose=0)
output: Wall time: 20 s
```

زمان محاسبه داده شده نکته مهمی را نشان می‌دهد. انتخاب شرایط اولیه ضعیف می‌تواند بر چگونگی همگرایی مدل‌ها، به ویژه مدل‌های خوشه‌ای تأثیر بگذارد. با MiniBatch k-means، هیچ تضمینی وجود ندارد که بهینه سراسری به دست آید. قدرت minibatch به خاطر استفاده از نمونه‌های تصادفی است، مانند bootstrapping. هنگام ایجاد یک الگوریتم برای داده‌های بزرگ، می‌توانید از نمونه‌های تصادفی زیاد در بسیاری از سیستم‌ها که به طور موازی پردازش می‌شوند، استفاده کرد.

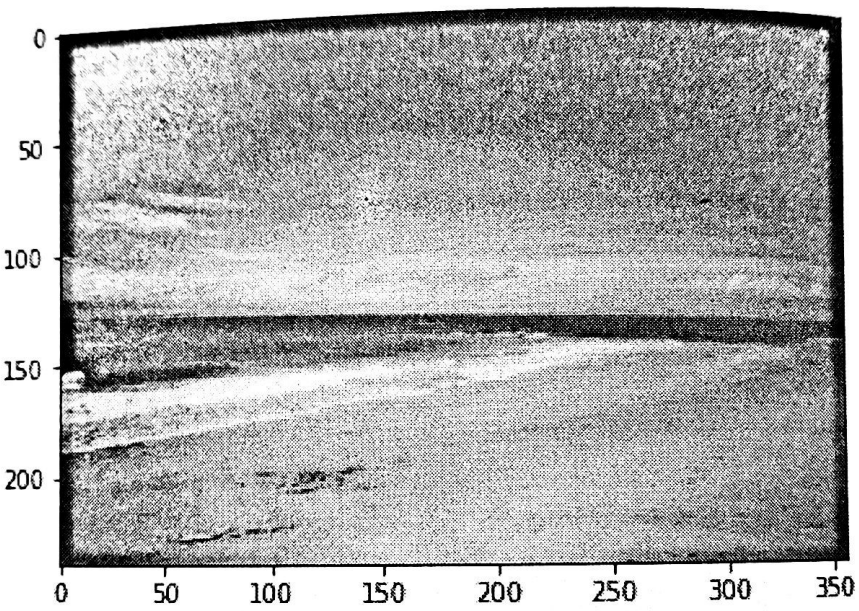
تصویر و خوشه‌بندی k-means

پردازش تصویر موضوع مهمی است که خوشه‌بندی در آن کاربردهای مختلفی دارد. شایان ذکر است که چندین کتابخانه پردازش تصویر بسیار خوب در پایتون وجود دارد. scikitimage یک پروژه خواهرانه از scikit است. یک نکته مهم در این فصل این است که تصاویر به عنوان داده هستند و می‌توان با استفاده از خوشه‌بندی حدس زد که برخی از اشیا در یک عکس کجا هستند. خوشه‌بندی می‌تواند بخشی از خط لوله پردازش تصویر باشد.

هدف استفاده از خوشه برای بخش‌بندی کردن تصویر است. ابتدا از SciPy برای خواندن تصویر استفاده خواهیم کرد. تصویر در یک آرایه سه بعدی ذخیره می‌شود. مختصات x و y بیانگر ارتفاع و عرض می‌باشند، و بعد سوم مقادیر RGB را برای هر تصویر نشان می‌دهد.

برای کوانتیزه کردن تصویر، باید آن را به یک آرایه دو بعدی تبدیل کنیم، طول آن برابر(طول تصویر در عرض تصویر) و عرض آن مقادیر RGB است. برای درک بهتر فرض کنید گروهی از نقاط با سه ویژگی داده شده‌اند (۳ ستون دارند).

```
%matplotlib inline
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
img = mpimg.imread("test.jpg")
plt.imshow(img)
```



شکل (۶-۶): تصویر اولیه

```
img.shape
output: (239, 357, 3)
x, y, z = img.shape
long_img = img.reshape(x*y, z)
long_img.shape
```

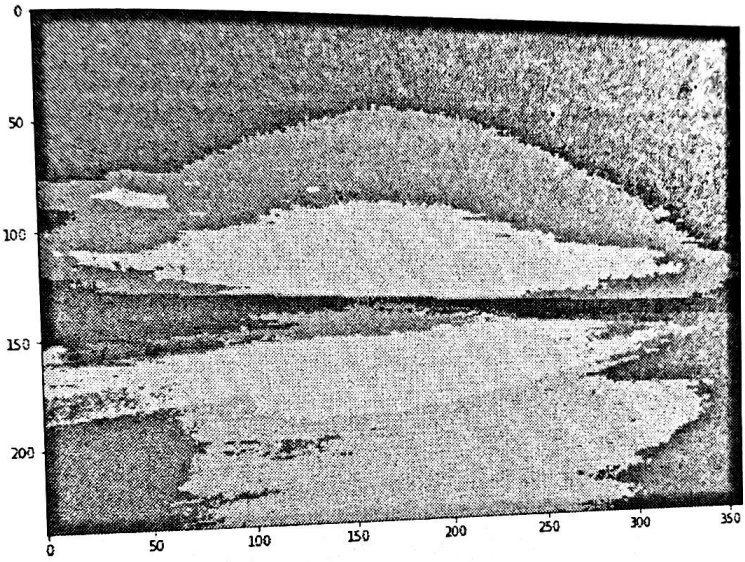
ما فرض می‌کنیم که ۵ خوشه داریم و عمل خوشه‌بندی را انجام می‌دهیم

```
from sklearn import cluster
k_means = cluster.KMeans(n_clusters=5)
k_means.fit(long_img)
centers = k_means.cluster_centers_
centers
output: array([[122.6965221 , 121.41881416, 108.33825686],
   [ 10.38406692, 105.94673679, 154.88697893],
   [ 80.14064045, 156.33290059, 189.67655786],
```

```
[178.8549281 , 172.08495265, 155.24538758],
[159.3334117 , 190.50614164, 210.16508963]])
labels = k_means.labels_
labels
output: array([1, 1, 1, ..., 0, 0, 0])
```

حال آرایه را به اندازه تصویر اولیه تبدیل می‌کنیم

```
plt.figure(figsize = (10,7))
plt.imshow(centers[labels].astype(int).reshape(x, y, z))
```



شکل (۶-۷): تصویر پس از خوشه‌بندی

یافتن نزدیک‌ترین شی در فضای ویژگی

گاهی اوقات، آسانترین کار این است که فاصله بین دو جسم را پیدا کنید. فقط باید معیار فاصله را پیدا کنیم، زوج فاصله هارا محاسبه کنیم و نتایج را با آنچه انتظار می‌رود مقایسه کنیم. یک ابزار سطح پایین در یادگیری sklearn.metrics.pairwise است. این کتابخانه شامل توابعی است که برای محاسبه فاصله بین بردارها در یک ماتریس X یا بین بردارهای X و Y به راحتی استفاده می‌شود. به عنوان مثال، با توجه به مجموعه‌ای از مشتریان با ویژگی‌های X، ممکن است بخواهیم مشتری مرجع بگیریم و نزدیک‌ترین مشتری را به این مشتری پیدا کنیم. در حقیقت، ممکن است بخواهیم مشتریان را با مفهوم شباهت که با تابع فاصله اندازه‌گیری می‌شود، درجه‌بندی کنیم. کیفیت شباهت به انتخاب فضای ویژگی و همچنین هر تحولی که ممکن است در فضا انجام دهیم بستگی دارد. ما چندین سناریو مختلف اندازه‌گیری مسافت را مرور خواهیم کرد.

برای تعیین نزدیکی اجسام از تابع pairwise_distanceces استفاده می‌شود. به یاد داشته باشید که نزدیکی فقط شباهت است، که ما آن را با استفاده از عملکرد فاصله خود درجه‌بندی می‌کنیم:

```
import numpy as np
from sklearn.metrics import pairwise
from sklearn.datasets import make_blobs
points, labels = make_blobs()
distances = pairwise.pairwise_distances(points)
```

فاصله یک ماتریس N × N است که مقادیر قطر آن صفر است است. در ساده‌ترین حالت، بیایید فواصل بین هر نقطه و نقطه اول را ببینیم

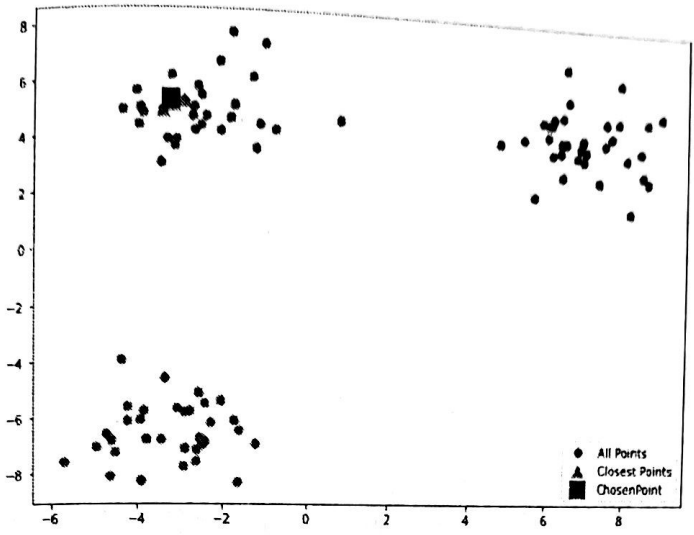
```
distances = pairwise.pairwise_distances(points)
np.diag(distances) [:5]
distances[0][:5]
output: array([ 0.   , 14.12336062, 2.9091087 , 4.65519048, 1.79535322])
```

رتبه‌بندی بر اساس نزدیکی با np.argsort انجام می‌دهیم

```
ranks = np.argsort(distances[0])
ranks[:5]
output: array([ 0, 96, 63, 24, 65], dtype=int64)
```

نکته جالب در مورد argsort این است که اکنون می‌توانیم ماتریس امتیازات شما را مرتب کنیم تا امتیاز واقعی را بدست آوریم

```
points[ranks][:5]
output: array([[5.19526954, 7.74605915],
   [5.08569441, 7.58001242],
   [5.27225053, 7.32184977],
   [4.82776352, 8.08981365],
   [5.80513633, 7.65300365]])
sp_points = points[ranks][:5]
import matplotlib.pyplot as plt
%matplotlib inline
plt.figure(figsize=(10,7))
plt.scatter(points[:,0], points[:,1], label = 'All Points')
plt.scatter(sp_points[:,0],sp_points[:,1],color='red',label='Closest Points')
plt.scatter(points[0,0],points[0,1],color='green', label = 'ChosenPoint')
plt.legend()
```



شکل (۶-۸): نزدیک‌ترین همسایگی‌ها

روش کار

با توجه به برخی از توابع فاصله، هر نقطه در یک تابع جفتی اندازه‌گیری می‌شود. دو نقطه را به عنوان بردار در فضای N بعدی با اجزای p در نظر بگیرید؛ به طور پیش فرض فاصله اقلیدسی در نظر گرفته شده است، که به شرح زیر است

$$d(p,q) = \sqrt{\sum_{1}^{N}(p_i - q_i)^2}$$

اختلاف بین هر یک از اجزای دو بردار را در نظر می‌گیرد، این تفاوت‌ها را به صورت مربع درآورده، همه آن‌ها را جمع می‌کند و سپس ریشه مربع را پیدا می‌کند. این بسیار آشنا به نظر می‌رسد زیرا ما هنگام مشاهده میانگین خطای مربع از چیزی بسیار مشابه استفاده کردیم در حقیقت، معیاری که اغلب استفاده می‌شود، میانگین ریشه انحراف مربع (RMSE) است که فقط تابع فاصله اعمال شده است.

محاسبه آن در پایتون به صورت زیر است

```
def euclid_distances(x, y):
 return np.power(np.power(x - y, 2).sum(), .5)
euclid_distances(points[0], points[1])
output: 14.123360622817486
```

اکنون می‌توانیم مسئله را حل کنیم. به عنوان مثال، اگر در مبدأ شبکه‌ای ایستاده بودیم و خطوط آن خیابان‌ها باشد، برای رسیدن به نقطه (۵، ۵) چه مقدار مسافت باید طی کنیم؟

```
pairwise.pairwise_distances([[0, 0], [5, 5]], metric='cityblock')[0]
output: array([ 0., 10.])
```

با استفاده از فاصله‌های دوتایی می‌توان به شباهت بین بردارهای بیتی پی برد. برای بردار N بعدی، این مسئله یافتن فاصله hamming است که به صورت زیر تعریف می‌شود:

$$d(p, q) = \sum_{1}^{N} I_{p_i \neq q_i}$$

```
X = np.random.binomial(1, .5, size=(2, 4)).astype(np.bool)
X
Output: array([[ True, True, False, True],
  [ True, True, True, False]])
pairwise.pairwise_distances(X, metric='hamming')
output: array([[0. , 0.5],
  [0.5, 0. ]])
```

خوشه‌بندی احتمالی با مدل‌های مخلوط گاوسی (Gaussian mixture models)

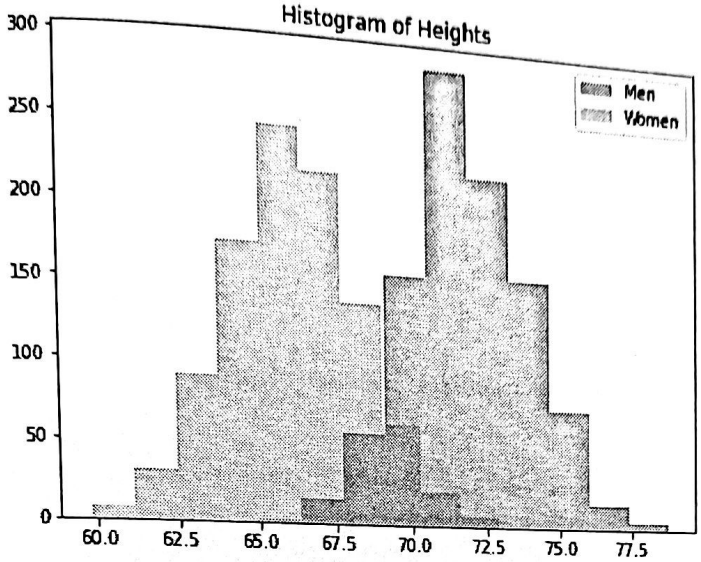
در k-means، ما تصور می‌کنیم که واریانس خوشه‌ها برابر است. این امر منجر به تقسیم فضا برای ایجاد خوشه‌ها می‌شود. اما در مورد وضعیتی که واریانس برابر نیستند چطور؟

خوشه‌بندی سخت k-mean مانند استفاده از یک مدل مخلوط گاوسی با یک ماتریس کوواریانس، S است که این همان ساختار کوواریانس برای هر خوشه است که منجر به خوشه‌های کروی می‌شود. با این حال، اگر S متفاوت باشد، می‌توان GMM را تخمین زده و برای پیش‌بینی استفاده کرد.

در ابتدا، ما باید برخی از داده‌ها را ایجاد کنیم. به عنوان مثال، بیایید قد زنان و مردان را شبیه‌سازی کنیم. از این مثال در کل این دستورالعمل استفاده خواهیم کرد.

```
import numpy as np
N = 1000
in_m = 72
in_w = 66
s_m = 2
s_w = s_m
m = np.random.normal(in_m, s_m, N)
w = np.random.normal(in_w, s_w, N)
from matplotlib import pyplot as plt
%matplotlib inline
f, ax = plt.subplots(figsize=(7, 5))
ax.set_title("Histogram of Heights")
```

```
ax.hist(m, alpha=.5, label="Men");
ax.hist(w, alpha=.5, label="Women");
ax.legend()
```



شکل (۶-۹): توزیع مقادیر قد بر اساس جنسیت

سپس به نمونه‌برداری از گروه، متناسب‌سازی توزیع و سپس پیش‌بینی گروه‌های باقیمانده می‌پردازیم:

```
random_sample = np.random.choice([True, False], size=m.size)
m_test = m[random_sample]
m_train = m[~random_sample]
w_test = w[random_sample]
w_train = w[~random_sample]
```

اکنون باید توزیع قد زنان و مردان را بر اساس مجموعه آموزش بدست آوریم. برای مجموعه آزمایش، ما از احتمال هر دو توزیع استفاده خواهیم کرد و به احتمال زیاد ، و توزیعی که شانس بیشتری دارد برای برچسب انتخاب می‌شود.

```
from scipy import stats
m_pdf = stats.norm(m_train.mean(), m_train.std())
w_pdf = stats.norm(w_train.mean(), w_train.std())
print(m_pdf.pdf(m[0]))
print(w_pdf.pdf(m[0]))
output: 0.10282407541039001
0.0552717604286798
```

به تفاوت احتمالات توجه کنید. شرایطی را فرض کنید که احتمال مردان بیشتر است، اما اگر احتمال زنان بیشتر باشد آن‌ها را بازنویسی می‌کنیم. بدیهی است که سوال این است

که ما چقدر دقیق هستیم. از آنجا که اگر guesses_m برابر ۱ در غیر این صورت برابر صفر است، میانگین بردار را می‌گیریم و دقت را بدست می‌آوریم.

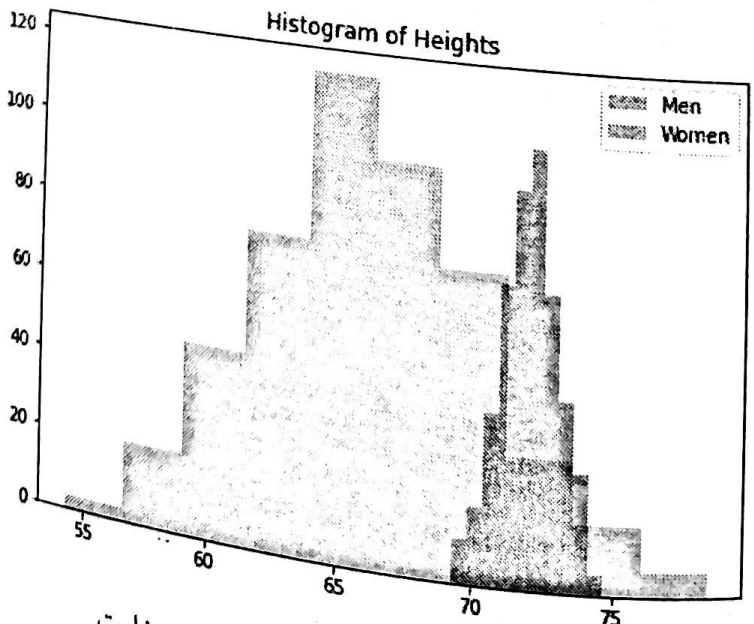
```
guesses_m = np.ones_like(m_test)
guesses_m[m_pdf.pdf(m_test) < w_pdf.pdf(m_test)] = 0
guesses_m.mean()
output: 0.9365079365079365
```

حالا، برای گروه خانم‌ها از دستورات زیر استفاده می‌کنیم

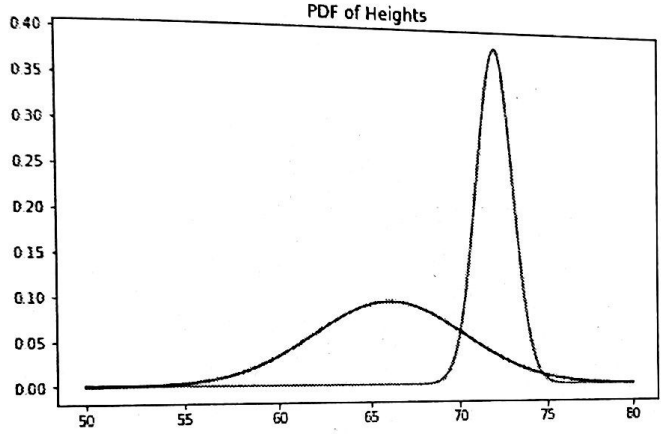
```
guesses_w = np.ones_like(w_test)
guesses_w[m_pdf.pdf(w_test) > w_pdf.pdf(w_test)] = 0
guesses_w.mean()
output: 0.9126984126984127
```

اجازه دهید واریانس بین گروهها متفاوت باشد. ابتدا چند داده جدید ایجاد کنید

```
s_m = 1
s_w = 4
m = np.random.normal(in_m, s_m, N)
w = np.random.normal(in_w, s_w, N)
m_test = m[random_sample]
m_train = m[~random_sample]
w_test = w[random_sample]
w_train = w[~random_sample]
f, ax = plt.subplots(figsize=(7, 5))
ax.set_title("Histogram of Heights")
ax.hist(m_train, alpha=.5, label="Men");
ax.hist(w_train, alpha=.5, label="Women");
ax.legend()
```



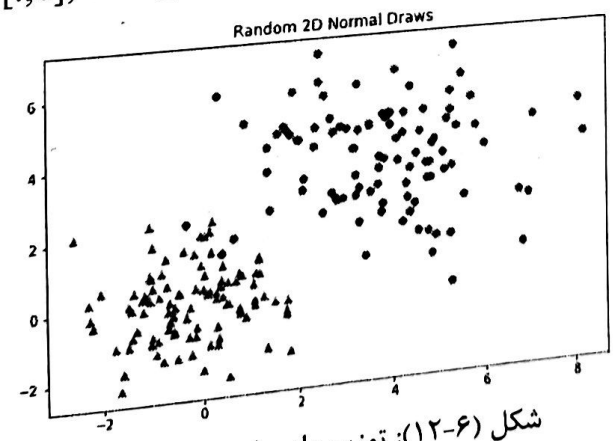
```
m_pdf = stats.norm(m_train.mean(), m_train.std())
w_pdf = stats.norm(w_train.mean(), w_train.std())
x = np.linspace(50,80,300)
plt.figure(figsize=(8,5))
plt.title('PDF of Heights')
plt.plot(x, m_pdf.pdf(x), 'k', linewidth=2, color='blue',
label='Men')
plt.plot(x, w_pdf.pdf(x), 'k', linewidth=2,
color='green',label='Women')
```



شکل (۶-۱۱): PDF توزیع قدها

تصور کنید در یک فضای چند بعدی قرار دارید:

```
class_A = np.random.normal(0, 1, size=(100, 2))
class_B = np.random.normal(4, 1.5, size=(100, 2))
f, ax = plt.subplots(figsize=(8, 5))
plt.title('Random 2D Normal Draws')
ax.scatter(class_A[:,0], class_A[:,1], label='A', c='r')
ax.scatter(class_B[:,0], class_B[:,1], label='B')
```



شکل (۶-۱۲): توزیع داده‌های دو کلاس

بیایید ببینیم که چگونه می‌توانیم نقاط را بر اساس توزیع، طبقه‌بندی کنیم

```
from sklearn.mixture import GaussianMixture
gmm = GaussianMixture(n_components=2)
X = np.row_stack((class_A, class_B))
y = np.hstack((np.ones(100), np.zeros(100)))
train = np.random.choice([True, False], 200)
gmm.fit(X[train])
GaussianMixture(covariance_type='full', init_params='kmeans', max_iter=100,
means_init=None, n_components=2, n_init=1, precisions_init=None,
random_state=None, reg_covar=1e-06, tol=0.001, verbose=0,
verbose_interval=10, warm_start=False, weights_init=None)
gmm.fit(X[train])
gmm.predict(X[train])[:5]
output: array([1, 1, 1, 1, 1], dtype=int64)
```

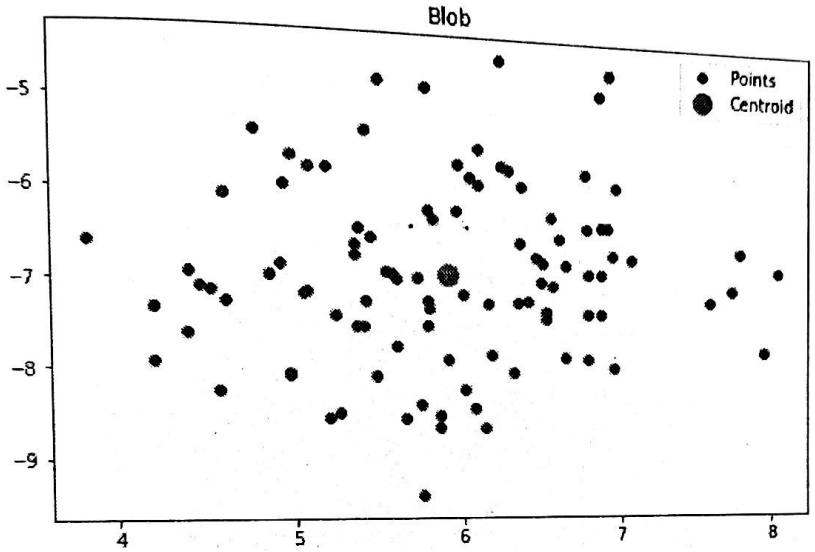
از k-means استفاده خواهیم کرد تا بتوانیم در یک خوشه از نقاط مختلف، داده‌های پرت را تشخیص دهیم. توجه به این نکته مهم است که بحث‌های زیادی برای داده‌های پرت و تشخیص آن‌ها وجود دارد. داده‌های پرت یا بر اثر خطای اندازه‌گیری یا عوامل خارجی ایجاد می‌گردند. ادامه مطلب پیرامون پیدا کردن داده‌های پرت است. ما با این فرض کار خواهیم کرد که انتخاب ما برای حذف داده‌های پرت موجه است زیرا آشکارسازی آن‌ها، در یافتن مرکز خوشه‌ها ضروری است.

ابتدا یک مجموعه داده با ۱۰۰ نقطه ایجاد خواهیم کرد؛ و سپس پنج نقطه دورتر از مرکز را شناسایی خواهیم کرد. (این موارد پرت احتمالی است)

مهم است که خوشه k-means دارای یک مرکز واحد باشد. این ایده شبیه SVM یک کلاسه است که برای تشخیص داده پرت استفاده می‌شود.

```
from sklearn.datasets import make_blobs
X, labels = make_blobs(100, centers=1)
import numpy as np
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=1)
kmeans.fit(X)
KMeans(algorithm='auto', copy_x=True, init='k-means++',
max_iter=300,
n_clusters=1, n_init=10, n_jobs=1, precompute_distances='auto',
random_state=None, tol=0.0001, verbose=0)
import matplotlib.pyplot as plt
%matplotlib inline
```

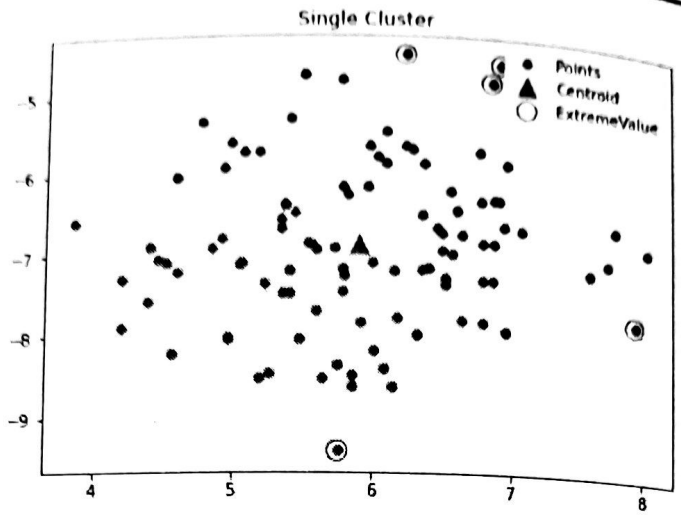
```
f, ax = plt.subplots(figsize=(8, 5))
ax.set_title("Blob")
ax.scatter(X[:, 0], X[:, 1], label='Points')
ax.scatter(kmeans.cluster_centers_[:, 0],kmeans.cluster_centers_[:,1],
label='Centroid',color='r')
ax.legend()
```



شکل (۶-۱۳): تشخیص مرکز کل داده ها

حال ۵ نقطه پرت را پیدا می‌کنیم

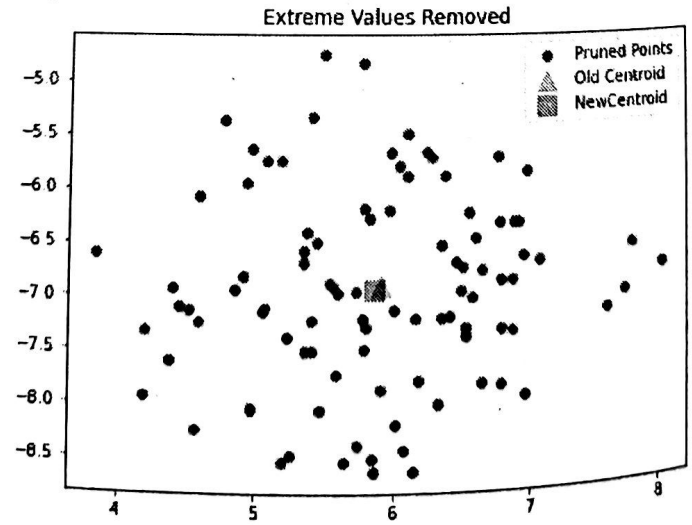
```
distances = kmeans.transform(X)
# argsort returns an array of indexes which will sort the array in ascending order
# so we reverse it via [::-1] and take the top five with [:5]
sorted_idx = np.argsort(distances.ravel())[::-1][:5]
f, ax = plt.subplots(figsize=(7, 5))
ax.set_title("Single Cluster")
ax.scatter(X[:, 0], X[:, 1], label='Points')
ax.scatter(kmeans.cluster_centers_[:,
0],kmeans.cluster_centers_[:,1],label='Centroid', color='r')
ax.scatter(X[sorted_idx][:, 0], X[sorted_idx][:, 1],label='ExtremeValue',
edgecolors='g',facecolors='none', s=100)
ax.legend(loc='best')
```



شکل (۶-۱۴): نقاط پرت نسبت به مرکز خوشه

حال اگر نیاز بود آن‌ها را حذف می‌کنیم، البته اگر دوباره مرکز را محاسبه نماییم تفاوت خواهد بود

```
new_X = np.delete(X, sorted_idx, axis=0)
new_kmeans = KMeans(n_clusters=1)
new_kmeans.fit(new_X)
f, ax = plt.subplots(figsize=(7, 5))
ax.set_title("Extreme Values Removed")
ax.scatter(new_X[:, 0], new_X[:, 1], label='Pruned Points')
ax.scatter(kmeans.cluster_centers_[:, 0],kmeans.cluster_centers_[:,1],
label='Old Centroid',color='r',s=80, alpha=.5)
ax.scatter(new_kmeans.cluster_centers_[:,
0],new_kmeans.cluster_centers_[:, 1], label='NewCentroid',color='m', s=80,
alpha=.5)
ax.legend(loc='best')
```



شکل(۶-۱۵): مقایسه مرکز قدیم با مرکز جدید خوشه

همانطور که قبلاً دیدیم، یک ارتباط اساسی بین توزیع گاوسی و خوشه‌بندی k-means وجود دارد. بیایید یک گاوسی بر اساس ماتریس کوواریانس مرکز و نمونه ایجاد کنیم و احتمال هر نقطه را بررسی کنیم (پنج نقطه را حذف کردیم). این نشان دهنده این است که مقادیر را با کمترین احتمال حذف کرده‌ایم. این ایده بین فواصل و احتمالات بسیار مهم است و اغلب در آموزش داده‌ها به کار می‌رود. از دستور زیر برای ایجاد Gaussian استفاده کنید

```
from scipy import stats
emp_dist = stats.multivariate_normal(kmeans.cluster_centers_.ravel())
lowest_prob_idx = np.argsort(emp_dist.pdf(X))[:5]
np.all(X[sorted_idx] == X[lowest_prob_idx])
output: True
```

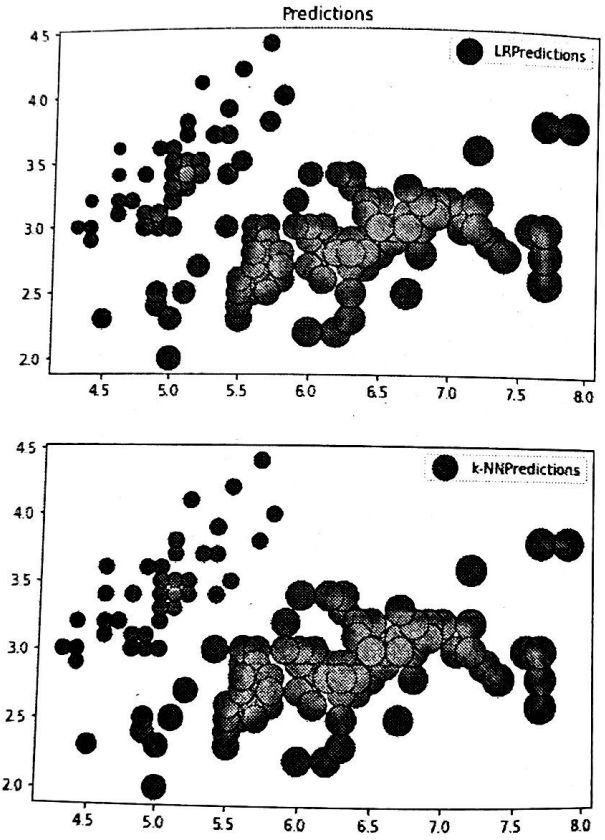
KNN برای رگرسیون

ما می‌توانیم تصور کنیم که مجموعه داده ما تحت چندین پردازش داده است. اگر این درست باشد، فقط آموزش روی نقاط داده مشابه ایده خوبی است. رگرسیون که یک تکنیک تحت نظارت است می‌تواند در زمینه خوشه‌بندی استفاده شود، بنابراین ما از خوشه‌بندی K-Nearest همسایه‌ها (KNN) به جای kmeans استفاده خواهیم کرد.

برای این دستورالعمل، ما از مجموعه داده iris استفاده خواهیم کرد. اگر بخواهیم چیزی مانند عرض گلبرگ را برای هر گل پیش‌بینی کنیم، خوشه‌بندی برای گونه‌های iris به طور بالقوه می‌تواند نتایج بهتری به ما بدهد.

```
import numpy as np
from sklearn import datasets
iris = datasets.load_iris()
iris.feature_names
X = iris.data[:,:2]
y = iris.data[:,2]
from sklearn.linear_model import LinearRegression
lr = LinearRegression()
lr.fit(X, y)
print ("The MSE is: {:.2}".format(np.power(y -lr.predict(X),2).mean()))
output: The MSE is: 0.41
from sklearn.neighbors import KNeighborsRegressor
knnr = KNeighborsRegressor(n_neighbors=10)
knnr.fit(X, y)
```

```
print ("The MSE is: {:.2}".format(np.power(y -knnr.predict(X),2).mean()))
output: The MSE is: 0.17
f, ax = plt.subplots(nrows=2, figsize=(7, 10))
ax[0].set_title("Predictions")
ax[0].scatter(X[:, 0], X[:, 1], s=lr.predict(X)*80, label='LRPredictions',
color='c', edgecolors='black')
ax[1].scatter(X[:, 0], X[:, 1], s=knnr.predict(X)*80, label='k-NNPredictions',
color='m', edgecolors='black')
ax[0].legend()
ax[1].legend()
```



شکل (۶-۱۶): شکل بالا پیش‌بینی رگرسیون لجستیک، شکل پایین پیش‌بینی KNN

```
setosa_idx = np.where(iris.target_names=='setosa')
setosa_mask = iris.target == setosa_idx[0]
y[setosa_mask][:5]
knnr.predict(X)[setosa_mask][:5]
output: array([1.46, 1.45, 1.51, 1.42, 1.46])
lr.predict(X)[setosa_mask][:5]
output: array([1.84557896, 2.1597721 , 1.53692893, 1.49323201, 1.53415738])
```

با نگاهی دوباره به تصاویرها، می‌بینیم که گونه setosa (خوشه فوقانی چپ) تا حد زیادی توسط رگرسیون خطی بیش از حد تخمین زده می‌شود و KNN تقریباً به مقادیر واقعی نزدیک است.

رگرسیون KNN با گرفتن میانگین K نزدیک‌ترین نقطه به نقطه آزمایش محاسبه می‌شود.

```
from sklearn.metrics import pairwise
distances_to_example = pairwise.pairwise_distances(X)[0]
ten_closest_points = X[np.argsort(distances_to_example)][:10]
ten_closest_y = y[np.argsort(distances_to_example)][:10]
ten_closest_y.mean()
output: 1.46
```